

Récurtivité - Correction

Warren McCulloch, avec Walter Pitts, est célèbre pour avoir posé les bases de la modélisation des neurones et de la logique formelle. Leur approche montrait que les réseaux de neurones simples pouvaient réaliser des fonctions logiques, et que certaines structures pouvaient simuler des processus récursifs. La récursivité, dans ce contexte, apparaît comme un mécanisme permettant à une structure simple de répéter des calculs ou de traiter des informations à plusieurs niveaux, inspirant plus tard les concepts d'algorithmes et de calculabilité.

Nicolas VACELET – Université Marie et Louis Pasteur

14 Octobre 2025

1 Théorème de Cormen-Leiserson-Rivest (Master Theorem)

Soit $a \geq 1, b > 1, f$ une fonction et $C(n)$ définie pour $n > n_0$ par :

$$C(n) = a C\left(\frac{n}{b}\right) + f(n)$$

alors :

1. Si $f(n) = O(n^{\log_b a - \varepsilon})$ pour une constante $\varepsilon > 0$, alors $C(n) = \Theta(n^{\log_b a})$
2. Si $f(n) = \Theta(n^{\log_b a})$, alors $C(n) = \Theta(n^{\log_b a} \log n)$

Exercice 1 : Fonction puissance

1. Nombre de multiplications pour puissance1 La fonction `puissance1` suit la récurrence simple :

$$M_1(n) = \begin{cases} 0 & n = 0, \\ 1 + M_1(n-1) & n > 0. \end{cases}$$

Donc le nombre de multiplications est exactement n :

$$n = 20 \rightarrow 20, \quad n = 200 \rightarrow 200, \quad n = 2000 \rightarrow 2000$$

2. Nombre de multiplications pour puissance2 `puissance2` utilise l'exponentiation rapide avec division par 2. La récurrence est :

$$M_2(n) = \begin{cases} 0 & n = 0, \\ M_2(\lfloor n/2 \rfloor) + 1 & n > 0. \end{cases}$$

Résolvons approximativement :

$$M_2(n) \approx \log_2(n) + 1$$

Pour nos valeurs :

$$n = 20 \rightarrow 5 \text{ ou } 6 \text{ multiplications}, \quad n = 200 \rightarrow 8, \quad n = 2000 \rightarrow 11$$

3. Analyse de complexité - `puissance1` : $O(n)$ en multiplications. - `puissance2` : $O(\log n)$ en multiplications.

La méthode exponentiation rapide réduit donc drastiquement le nombre d'opérations pour des grands n .

4. Comparaison `puissance2` est nettement plus efficace pour n grand. `puissance1` devient très lente dès que n croît.

5. Puissance3 : recalcul double

$$\text{puissance3}(x, n) = \begin{cases} 1 & n = 0, \\ \text{puissance3}(x, \lfloor n/2 \rfloor) \cdot \text{puissance3}(x, \lfloor n/2 \rfloor) & n > 0 \text{ pair}, \\ x \cdot \text{puissance3}(x, \lfloor n/2 \rfloor) \cdot \text{puissance3}(x, \lfloor n/2 \rfloor) & n > 0 \text{ impair}. \end{cases}$$

Ici, chaque appel effectue deux sous-appels au lieu d'un seul. La récurrence devient :

$$M_3(n) = 2M_3(\lfloor n/2 \rfloor) + 1$$

Résolution approximative avec le théorème du Master (cas $a = 2, b = 2, f(n) = 1$) :

$$\log_b a = \log_2 2 = 1, \quad f(n) = O(n^0)$$

On est dans le cas 1 du Master Theorem : $M_3(n) = \Theta(n)$

Conclusion : **cette "optimisation" est en fait catastrophique**, car elle double le nombre d'appels récurifs et rend l'algorithme beaucoup moins efficace que puissance2.

$$\text{puissance4}(x, n) = \begin{cases} 1 & n = 0, \\ (\text{puissance4}(x, \lfloor n/2 \rfloor))^2 & n > 0 \text{ pair}, \\ x \cdot (\text{puissance4}(x, \lfloor n/2 \rfloor))^2 & n > 0 \text{ impair}. \end{cases}$$

Exercice 2 : Tours de Hanoï

1. Solution pour petits n

- $n = 1$: $A \rightarrow B$
- $n = 2$: $A \rightarrow C, A \rightarrow B, C \rightarrow B$
- $n = 3$: $A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow B, C \rightarrow A, C \rightarrow B, A \rightarrow B$
- $n = 4$: déplacer les 3 premiers sur C, déplacer le plus grand sur B, déplacer les 3 de C sur B $\rightarrow$ total 15 mouvements

2. Algorithme récursif

$$\text{hanoi}(n, \text{source}, \text{cible}, \text{auxiliaire}) = \begin{cases} \text{Déplacer disque de source vers cible,} & \text{si } n = 1 \\ \left\{ \begin{array}{l} \text{hanoi}(n-1, \text{source}, \text{auxiliaire}, \text{cible}) \\ \text{Déplacer disque de source vers cible} \\ \text{hanoi}(n-1, \text{auxiliaire}, \text{cible}, \text{source}) \end{array} \right. & \text{sinon} \end{cases}$$

3. Analyse du nombre de mouvements Récurrence :

$$T(n) = 2T(n-1) + 1, \quad T(1) = 1$$

Résolution :

$$T(n) = 2^n - 1$$

Donc le nombre de mouvements croît exponentiellement avec n . Pour $n = 10$, il faut 1023 mouvements ; pour $n = 20$, plus d'un million.